

// Jivops

Guía completa — 30 lecciones

Arquitectura · Topics y particionamiento · Producers y Consumers ·
Connect y Streams · Operación a escala · Producción avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Kafka y el modelo de streaming de eventos
- Lección 02 — Arquitectura — brokers, topics, partitions
- Lección 03 — Instalación de Kafka (KRaft mode, sin Zookeeper)
- Lección 04 — Productores y consumidores — conceptos básicos
- Lección 05 — Kafka CLI — comandos fundamentales

Módulo 2: Topics y particionamiento

- Lección 06 — Topics — creación y configuración
- Lección 07 — Particiones — cómo se distribuyen los datos
- Lección 08 — Replication factor — tolerancia a fallos
- Lección 09 — Claves de partición y estrategias de particionamiento
- Lección 10 — Retención de datos — time-based y size-based

Módulo 3: Producers y Consumers en profundidad

- Lección 11 — Configuración de producers — acks, batching
- Lección 12 — Idempotencia y exactly-once semantics
- Lección 13 — Consumer groups — escalado horizontal de consumo
- Lección 14 — Offsets — commit manual vs automático
- Lección 15 — Rebalancing de consumer groups

Módulo 4: Kafka Connect y Streams

- Lección 16 — Kafka Connect — arquitectura de conectores
- Lección 17 — Source y Sink connectors
- Lección 18 — Kafka Streams — procesamiento de streams
- Lección 19 — ksqlDB — SQL sobre streams
- Lección 20 — Schema Registry — gestión de esquemas Avro/Protobuf

Módulo 5: Operación a escala

- Lección 21 — Tuning de rendimiento — throughput y latencia
- Lección 22 — Monitoreo de Kafka con Prometheus (JMX exporter)
- Lección 23 — Gestión de partition reassignment
- Lección 24 — Backup y disaster recovery (MirrorMaker)
- Lección 25 — Troubleshooting de consumer lag

Módulo 6: Producción avanzada

Lección 26 — Seguridad — SASL, TLS, ACLs

Lección 27 — Kafka en Kubernetes (Strimzi)

Lección 28 — Multi-cluster y replicación geo-distribuida

Lección 29 — Actualización de Kafka sin downtime

Lección 30 — Proyecto final: plataforma de streaming completa con Kafka lista para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Kafka y el modelo de streaming de eventos

Como los eventos no se eliminan al ser leídos, un consumer nuevo puede posicionarse desde el inicio y leer todo el historial retenido.

Lección 02 — Arquitectura — brokers, topics, partitions

Distribuir un topic en partitions entre distintos brokers permite escrituras en paralelo, evitando que un único broker sea cuello de botella.

Lección 03 — Instalación de Kafka (KRaft mode, sin Zookeeper)

Con KRaft, toda la responsabilidad de consenso vive dentro del propio Kafka, reduciendo a un único sistema que operar en vez de dos.

Lección 04 — Productores y consumidores — conceptos básicos

El producer sigue generando eventos aunque el consumer esté caído; quedan retenidos esperando a que vuelva a estar disponible.

Lección 05 — Kafka CLI — comandos fundamentales

'--from-beginning' permite verificar rápidamente el contenido de un topic sin escribir y desplegar un consumer real completo.

```
kafka-console-producer --topic mi-topic
```

Módulo 2: Topics y particionamiento

Lección 06 — Topics — creación y configuración

Aumentar el número de partitions después puede afectar el orden garantizado dentro de una misma clave, por eso conviene planificarlo con cuidado.

```
--replication-factor 3
```

Lección 07 — Particiones — cómo se distribuyen los datos

Kafka acepta el tradeoff de no garantizar orden global entre partitions a cambio del paralelismo real que ofrece procesarlas independientemente.

Lección 08 — Replication factor — tolerancia a fallos

Sin réplicas adicionales, la falla del único broker que aloja esa partition dejaría los datos completamente inaccesibles hasta su recuperación.

Lección 09 — Claves de partición y estrategias de particionamiento

Usar el ID de usuario como key garantiza que sus eventos lleguen siempre a la misma partition, preservando el orden entre ellos.

Lección 10 — Retención de datos — time-based y size-based

Un consumer caído más tiempo del que dura la retención configurada pierde esos eventos, ya que se eliminan antes de que vuelva a estar disponible.

Módulo 3: Producers y Consumers en profundidad

Lección 11 — Configuración de producers — acks, batching

'acks=all' añade latencia esperando confirmación completa, un tradeoff que vale la pena para datos donde perder un evento sería inaceptable.

```
acks=0
```

Lección 12 — Idempotencia y exactly-once semantics

Sin idempotencia, un reintento tras perder la confirmación (no la escritura) podría duplicar un evento que en realidad sí se había escrito.

Lección 13 — Consumer groups — escalado horizontal de consumo

Más instancias que partitions no aumentan el throughput; las adicionales quedan sin ninguna partition asignada, sin aportar nada real.

Lección 14 — Offsets — commit manual vs automático

Con commit manual tras confirmar el éxito, un fallo previo al commit permite reprocesar el evento en vez de perderlo silenciosamente.

Lección 15 — Rebalancing de consumer groups

Rebalances frecuentes por instancias inestables acumulan pausas repetidas que degradan el throughput efectivo del grupo con el tiempo.

Módulo 4: Kafka Connect y Streams

Lección 16 — Kafka Connect — arquitectura de conectores

Un conector ya probado por la comunidad evita escribir y mantener manualmente la integración, un trabajo que se repetiría por cada sistema distinto.

Lección 17 — Source y Sink connectors

Un conector CDC lee el log de transacciones en tiempo real, mucho más eficiente que consultar periódicamente toda la tabla buscando cambios.

Lección 18 — Kafka Streams — procesamiento de streams

Sin necesitar un cluster de procesamiento separado como Spark, la lógica vive dentro de la propia aplicación, simplificando el despliegue.

Lección 19 — ksqlDB — SQL sobre streams

Un analista familiarizado con SQL puede contribuir directamente a la lógica de streaming sin aprender Java y la API de Kafka Streams.

Lección 20 — Schema Registry — gestión de esquemas Avro/Protobuf

Sin esa validación, un cambio incompatible de esquema solo se descubriría cuando los consumers existentes empiecen a fallar en producción.

Módulo 5: Operación a escala

Lección 21 — Tuning de rendimiento — throughput y latencia

Notificaciones en tiempo real prefieren `linger.ms` bajo; ingesta masiva de logs se beneficia de uno más alto para maximizar el throughput.

```
linger.ms=10
```

Lección 22 — Monitoreo de Kafka con Prometheus (JMX exporter)

Un aumento de 'under-replicated partitions' es una señal temprana de degradación de redundancia, antes de una pérdida real de disponibilidad.

Lección 23 — Gestión de partition reassignment

Sin reasignación manual, un broker nuevo quedaría completamente sin carga, desaprovechando la capacidad adicional que se pretendía ganar.

```
kafka-reassign-partitions
```

Lección 24 — Backup y disaster recovery (MirrorMaker)

Solo un cluster réplica en otra región geográfica, sincronizado con MirrorMaker, sobreviviría a un desastre que afecte a toda una región.

Lección 25 — Troubleshooting de consumer lag

Un lag que crece de forma sostenida (no estable) señala que el consumer necesita escalar o su procesamiento necesita optimizarse.

```
kafka-consumer-groups --describe --group mi-grupo
```


Módulo 6: Producción avanzada

Lección 26 — Seguridad — SASL, TLS, ACLs

TLS protege el contenido en tránsito, pero sin SASL cualquiera con conectividad de red podría conectarse sin verificar su identidad.

Lección 27 — Kafka en Kubernetes (Strimzi)

Definir Kafka con CRDs declarativos permite versionarlo y auditarlo mediante el mismo flujo GitOps del resto del cluster.

Lección 28 — Multi-cluster y replicación geo-distribuida

Escribir hacia un cluster centralizado lejano añade latencia proporcional a la distancia, algo que un cluster local en la misma región evita.

Lección 29 — Actualización de Kafka sin downtime

Actualizar el siguiente broker antes de completar la resincronización arriesga dejar al cluster con menos redundancia durante el proceso.

Lección 30 — Proyecto final: plataforma de streaming completa con Kafka lista para producción

Se integra todo lo anterior: topics bien particionados con replication factor adecuado, garantías correctas en producers/consumers, seguridad SASL/TLS/ACLs, monitoreo de consumer lag y una estrategia de disaster recovery — el nivel esperado de una plataforma de streaming con Kafka lista para producción real.